

# Apunte de la unidad 1:

## Proceso de desarrollo

### Programación con Objetos 1



## El proceso de desarrollo de software

El estudio de los conceptos del paradigma de objetos no suele bastar para enfrentarse a un problema de programación y resolverlo, pues la construcción de un programa se realiza tomando decisiones que definen su estructura. En efecto, elegir qué objetos aparecen en la solución, los mensajes involucrados, cuál es el estado interno de cada objeto y cómo este varía son algunas de las decisiones que se deben tomar. Todo este conjunto de ideas que se plasman en el código se denomina diseño de software. Y la manera en que estas decisiones se toman se conoce como **proceso de diseño**, que forma parte de un proceso más amplio: el del desarrollo. El proceso de desarrollo, en su conjunto, incluye otras actividades, entre las cuales se pueden mencionar: la detección de requerimientos, la evaluación de calidad y la puesta en producción.

En la disciplina del desarrollo tecnológico (y en particular del desarrollo de software) existen distintas metodologías de trabajo que plantean grandes diferencias en cuanto a la temporalidad de las tareas de diseño y codificación. En algunas metodologías (como la metodología de desarrollo en cascada), se separa forzosamente el momento de tomar todas las decisiones y de pensar las abstracciones (etapa de diseño) del momento de escribir el código de la solución que implementa dichas decisiones. Este tipo de abordaje presenta importantes desventajas. Por un lado, se avanza excesivamente en los detalles, muchos de los cuales pueden desestimarse en la etapa de programación por impracticidad o inaplicabilidad. Por otro lado, reduce la flexibilidad para elegir en qué momento ocuparse del diseño y en qué momento de la programación del código.

Por este motivo, se propone una metodología que combina las tareas antedichas de manera que se aborden de forma intercalada y en muchos momentos superpuestas. Es un proceso iterativo e incremental que se define en términos de las siguientes tareas:

- Se piensa acerca de un requerimiento o tarea,
- Se elige el mensaje y el objeto por donde comenzar

- A medida que se programa el método, se van realizando **preguntas** que guían la solución.
- Algunas respuestas a dichas preguntas, o simplemente por haber tomado decisiones no acertadas previamente, obligan a repensar y a cambiar tanto el código como las decisiones previamente tomadas. Es decir, el diseño del software se va modificando mientras se programa.
- Al finalizar el requerimiento o la tarea, se integra el código al repositorio y queda listo para iniciar una nueva funcionalidad.
- El código ya integrado también es susceptible de ser modificado posteriormente debido a un cambio en el diseño.

En este esquema, la mejor manera de comprobar que el diseño pensado funciona es desarrollar prototipos evolutivos que van formando la solución final. Un buen diseño produce código que funciona, y cuanto antes se cuente con código que implemente el diseño, más rápidamente pueden solucionarse los problemas surgidos por decisiones inadecuadas. El concepto de prototipo evolutivo se refiere a que la primera versión del programa no es final, ni siquiera es completa. Se podría ver como una *demo*. No es una *demo* que prueba un concepto y se desecha, sino que sobre esa implementación se sigue trabajando, construyendo y modelando hasta obtener una versión funcional completa.

Si bien el proceso implica tener el código como resultado principal, ello no significa que otras herramientas, como los diagramas, sean obsoletas. Por momentos conviene apoyarse en diagramas para representar o decidir las relaciones entre los componentes, o para discutir la solución con algún/a compañero/a.

A continuación se plantea un problema muy reducido cuyo objetivo es ofrecer una aproximación al proceso de desarrollo para plantear algunas de las preguntas más importantes que guían esta actividad. Sin embargo, no tiene como objetivo abarcar el proceso completo, porque, al igual que en el desarrollo de software, aprender a diseñar también es una tarea iterativa e incremental que va más allá de un documento introductorio.

## Ejemplo: Macacha<sup>1</sup>

El enunciado del problema es el siguiente:

Macacha realiza instalaciones para la empresa proveedora de servicios de internet Qnet.

Todos los meses, Machaca cobra una comisión total dependiendo de la cantidad de instalaciones que ha desarrollado:

```
...
```

```
comisionTotal = instalaciones * comisionPorInstalacion
```

```
...
```

Se pide saber la comisión total de Macacha para el mes en curso.

---

<sup>1</sup> [https://es.wikipedia.org/wiki/Macacha\\_G%C3%BCemes](https://es.wikipedia.org/wiki/Macacha_G%C3%BCemes)

## Detección de requerimientos

El primer paso es la interpretación del enunciado, una actividad similar a lo que en la industria se conoce como relevamiento y análisis funcional. En este momento se realiza un intercambio con las personas que conocen el dominio para identificar los requerimientos, entendiendo por requerimiento un problema que el sistema debe resolver ante un estímulo particular. Generalmente, ese estímulo es una acción de una persona sobre el sistema.

En el ejemplo, el primer requerimiento es muy directo: conocer la comisión que le corresponde a Macacha; queda claro que es el principal objetivo del sistema. Existen más requerimientos que podrían definirse en este instante, pero así como se propone la iteratividad en el desarrollo, también ocurre con la definición de las reglas de negocio. Se admite que **durante el proceso de diseño/desarrollo se recurre a analizar nuevamente el dominio, cambiando o ampliando los requerimientos.**

Es importante aclarar que esta materia se centra en resolver el dominio, es decir, la lógica de negocio. Por lo tanto, no forma parte de lo que se aborda aquí lo que se refiere a convertir la acción de una persona en el mensaje que finalmente se dirige a un objeto del dominio .

## Primer requerimiento: comisión de Macacha

El primer requerimiento es saber cuál es la comisión de Macacha correspondiente al mes en curso. La pregunta que ayuda a comenzar con las definiciones es: **¿Qué mensaje se debe enviar a qué objeto para resolver este requerimiento?** Toda solución comienza eligiendo un objeto y un mensaje (con sus posibles parámetros) para dicho objeto, que debe responder para cumplir el requerimiento. Esto puede dar lugar a la definición de nuevos objetos.

Los objetos candidatos que surgen son: un objeto para Macacha y, quizás, otro para modelar la empresa Qnet. Se debe optar por la solución más simple y directa: si la comisión es de Machaca, lo mejor es interactuar directamente con el objeto que la representa, poniendo allí la **responsabilidad**. El nombre del mensaje puede ser comision() y, en principio, no debería pasarse más información como parámetro. Es decir, las cosas de las que depende la comisión total de Macacha (las instalaciones y la comisión por instalación) corresponden a información que el sistema en sí ya debería conocer. Sería poco útil tener que definir esos valores al calcular la comisión total. Esto sugiere que el sistema tendrá que admitir más requerimientos para administrar esta información, lo cual se abordará más adelante.

Por otro lado, la decisión sobre la necesidad de parámetros se toma cada vez que se define un mensaje y el correspondiente método. En este caso, el mensaje comision() no requiere parámetros.

```
object macacha {  
  method comision() { //TO-DO  
}
```

```
}
```

La manera en que se usa este mensaje es como sigue:

```
wollok:example> macacha.comision()
```

## Programación de los métodos

Una vez definidos el objeto y el mensaje, se procede a escribir el código del método. En este punto se deben plantear dos interrogantes: ¿Es una orden o una consulta? ¿Se debe calcular o recordar? En estas situaciones puede ser necesario recurrir a quien conoce la lógica del negocio para entender mejor cómo debe funcionar el sistema, lo que fortalece lo dicho en cuanto al proceso iterativo e incremental. Esta persona puede ser el docente (o el enunciado de un ejercicio) en el marco de la cursada, o la persona analista funcional en caso de un proceso industrial.

### Orden o consulta

Es fundamental saber la respuesta: si es una orden, no tendrá valor de retorno y tendrá efecto, mientras que si es una consulta, tendrá un valor de retorno mediante la instrucción *return* y no debe tener efecto. En el ejemplo que se desarrolla aquí se espera que la comisión total sea una consulta.

```
object macacha {  
  method comisionTotal() {  
    return //TODO  
  }  
}
```

### Cálculo o atributo

Dentro del subconjunto de las consultas, se tiene dos grandes grupos de problemas:

- Aquellos que requieren de un cálculo: hay un algoritmo que tiene que procesar información
- Aquellos que deben exponer algo que se recuerda: Ya se tiene la respuesta y se la conoce mediante una referencia..

La pregunta **¿Es algo que se calcula?** puede realizarse en este momento para identificar el tipo de método que se va a escribir. Si la respuesta es *sí*, el cálculo se realiza en el cuerpo de este método. En este caso se realiza en función de las instalaciones y de la comisión por instalación, por lo tanto abstraemos en subtarefas para luego realizar las mismas preguntas con las que se resolvió la comisión total:

```
object macacha {  
  method comisionTotal() {  
    return self.instalaciones() * self.comisionPorInstalacion()  
  }  
  method instalaciones() { //TO DO  
  }  
  method comisionPorInstalacion() { //TO DO  
  }  
}
```

## Requerimiento 1: instalaciones

El aspecto de las instalaciones planteado en el ejercicio de ejemplo también merece plantear las preguntas de la sección anterior. Ante la pregunta sobre el tipo de mensaje (orden vs consulta), la respuesta es simple: se usa en `comisionTotal()` para devolver un valor; por lo tanto, estamos en la órbita de las consultas (habrá un *return*). Corresponde entonces analizar si se debe calcular o recordar la cantidad de instalaciones, y se ve claramente que es un dato que debe recordarse, ya que se trata de información que el sistema debe mantener y no puede generar automáticamente.

Así se define que, a medida que Macacha genera instalaciones, usa el sistema para informar de la nueva instalación. El pensar en cómo se utiliza el sistema para administrar el número de instalaciones de Macacha determina, en este caso, un nuevo requerimiento: informar que Macacha realizó una nueva instalación. Sin embargo, este es otro requerimiento que se abordará en un siguiente paso.

Una vez decidido que es algo que se debe recordar y, por ende, será un atributo en un objeto, la pregunta que se debe contestar es: **¿Quién debe recordar?** Naturalmente se infiere que si las instalaciones son de Macacha, y es información que necesita Macacha, corresponde tener un atributo ese objeto. En el objeto Macacha se decide asumir la responsabilidad de recordar cuáles son sus instalaciones.

El siguiente paso es determinar el tipo de atributo, para lo que se debe considerar es si **este varía o es siempre el mismo** y la respuesta determina si se trata de una variable o una constante. En este caso varía, por lo que se utiliza *var*. El enunciado no hace referencia a un buen valor inicial, pero intuitivamente corresponde utilizar 0 ya que instalaciones es un contador.

```
object macacha {  
  var instalaciones = 0  
  method comision() {
```

```

        return self.instalaciones() * self.comisionPorInstalacion()
    }

    method instalaciones() {
        return instalaciones
    }

    method comisionPorInstalacion() { //TODO
    }
}

```

En este momento es posible continuar con el requerimiento de la comisión por instalación o bien abordar el mecanismo para registrar las instalaciones realizadas. En la siguiente sección se aborda este segundo requerimiento.

## Requerimiento 2: Instalar (orden)

Ante la pregunta: **¿Es una orden o una consulta?** La respuesta es que es una orden porque se solicita al sistema que modifique lo que recuerda sobre las instalaciones de Macacha. La segunda pregunta que se hace en este caso es:

- **¿Hay algún efecto en el estado interno de este objeto?** Si la respuesta es sí, se altera algún atributo propio
- **¿Hay efecto en otro objeto?** Si se espera que el efecto ocurra dentro de algún objeto referenciado en el estado interno o en algún parámetro, entonces se debe enviar otra orden.

En este caso, sabemos que el efecto se limita al estado interno de Macacha, modificando el atributo instalaciones en el método *instalar()*.

```

object macacha {
    var instalaciones = 0

    method instalar() {
        instalaciones = instalaciones + 1
    }

    method comisionTotal() {
        return self.instalaciones() * self.comisionPorInstalacion()
    }

    method instalaciones() {
        return instalaciones
    }
}

```

```
}  
  
method comisionPorInstalacion() { //TODO  
  
}  
  
}
```

### Requerimiento 3: comisión por Instalación (consulta recordada)

Al igual que en requerimiento 1 -*instalaciones()*-, se detecta que *comisionPorInstalacion()* es una consulta, ya que se necesita el valor de retorno. Por otro lado, dado que se el valor retornado es algo que se debe recordar en el sistema -porque no es calculado-, no necesita parámetros.

La pregunta **¿quién debe recordarlo?** puede tener dos respuestas según el enunciado / experto de negocio, que se analizan en las siguientes secciones.

#### Comisión por Instalación (consulta recordada por macacha)

Si la comisión es específica para Macacha, lo más natural y lógico es que sea ésta quien lo recuerde. Luego la pregunta **¿puede variar?** No tiene una respuesta clara según este enunciado. Sería bueno consultar con quien sabe del negocio para determinarlo. Si puede variar será una variable, en caso contrario una constante. De la misma manera sería bueno establecer un valor inicial útil. En caso de que varíe, coordinar la mejor manera de configurar este valor. En este caso, la manera más natural parece ser un setter.

```
object macacha {  
  
  var comisionPorInstalacion = 100  
  
  var instalaciones = 0  
  
  method instalar() {  
  
    instalaciones = instalaciones + 1  
  
  }  
  
  method comision() {  
  
    return self.instalaciones() * self.comisionPorInstalacion()  
  
  }  
  
  method instalaciones() {  
  
    return instalaciones  
  
  }  
  
  method comisionPorInstalacion() {  
  
    return comisionPorInstalacion  
  
  }  
  
}
```

```
}  
  
method comisionPorInstalacion(_comisionPorInstalacion) {  
  
    comisionPorInstalacion = _comisionPorInstalacion  
  
}  
  
}
```

## Comisión por Instalación (consulta delegada)

Una variante que podría surgir es que la comisión por instalación es la misma para todas las personas que realizan el trabajo de instalación, en cuyo caso podría tener sentido que el valor lo recuerde un objeto externo que se relacione con todas ellas. A partir de analizar este dominio y considerar objetos candidatos, se puede modelar el *departamento de servicios de la empresa qNet*, aunque hay muchos otros nombres que este segundo objeto podría tener. Lo interesante es que se le asigna la responsabilidad de mantener la comisión de instalación de todas las personas que instalan.

En ese caso, es necesario establecer la relación entre Macacha y el departamento de servicios, y existen dos maneras de que dos objetos se conozcan: por atributo o por parámetro. Para decidir de qué forma hacerlo, se debe considerar la temporalidad (o duración) de esa relación entre los objetos: si se trata de algo eventual o duradero. Una pregunta que ayuda a resolver la cuestión es: **¿Esta relación se debe establecer para el momento puntual de la llamada o es algo que debe perdurar en el tiempo?** Si la relación es puntual para una llamada (eventual), corresponde utilizar un **parámetro**, mientras que si la relación es duradera, queriendo utilizar el mismo objeto entre las distintas llamadas, lo más conveniente es un **atributo**.

La relación entre Macacha y el departamento de servicios es duradera. Sería incómodo establecer por parámetro el departamento cada vez que se quiera consultar por la comisión por instalación. Para establecer el nombre del atributo, la pregunta guía es: **¿Cuál es el rol que el departamento de servicios está cumpliendo para Macacha?** Se define que el rol es el de ser el departamento al que pertenece, siendo simplemente *departamento* un buen nombre para el mismo. Para la inicialización se utiliza la referencia global generada por la estructura *object{}*.

```
object macacha {  
  
    const departamento = servicios  
  
    var instalaciones = 0  
  
    method instalar() {  
  
        instalaciones = instalaciones + 1  
  
    }  
  
    method comision() {  
  
        return self.instalaciones() * self.comisionPorInstalacion()  
  
    }  
  
}
```

```

}

method instalaciones() {
    return instalaciones
}

method comisionPorInstalacion() {
    return departamento.comisionPorInstalacion()
}
}

object servicios {
    var comisionPorInstalacion = 100
    method comisionPorInstalacion() {
        return comisionPorInstalacion
    }
    method comisionPorInstalacion(_comisionPorInstalacion) {
        comisionPorInstalacion = _comisionPorInstalacion
    }
}

```

## Simplificación

Una vez resuelto el problema, podría ser conveniente ver si quedan métodos *pasamanos* que son innecesarios. Por ejemplo, los métodos *comisionPorInstalacion()* e *instalaciones()* son subtareas que sirvieron para pensar la solución. Sin embargo, al analizar el código final se observa que no aporta utilidad. Alguien con experiencia incluso podría haber omitido ese paso de generar las subtareas, pero mientras se está aprendiendo es algo recomendable primero dividir en subtareas y luego mejorar la solución reestructurando el código sin que cambie su funcionalidad (*refactoring*<sup>2</sup>). Haber definido un atributo de manera temprana puede sesgar las decisiones futuras, dificultando que aparezcan métodos necesarios.

```

object macacha {
    const departamento = servicios
    var instalaciones = 0
    method instalar() {
        instalaciones = instalaciones + 1
    }
}

```

---

<sup>2</sup> <https://es.wikipedia.org/wiki/Refactorizaci%C3%B3n>

```
}  
  
method comision() {  
    return instalaciones * departamento.comisionPorInstalacion()  
}  
  
}
```

## Mala práctica: precálculo

El proceso de diseño desarrollado en este apunte propone iniciar la solución a partir de los requerimientos. Al trabajar sobre el método *comisionTotal()*, lo primero que se resuelve es si se trata de un cálculo o de algo que debe recordar. Al resolverse como un cálculo, se escribe el algoritmo y se avanza sin problemas.

Sin embargo, muchas veces sucede que la inexperiencia de quien programa ignora este consejo y realiza un camino distinto, en el cual decide que el programa debe recordar la comisión total en vez de calcularla. Esto trae como consecuencia que el dato debe ser actualizado cada vez que se produzca una instalación:

```
object macacha {  
    var comisionPorInstalacion = 100  
    var comisionTotal = 0  
    method instalar() {  
        comisionTotal = comisionTotal + self.comisionPorInstalacion()  
    }  
    method comisionTotal() {  
        return comisionTotal  
    }  
    method comisionPorInstalacion() {  
        return comisionPorInstalacion  
    }  
    method comisionPorInstalacion(_comisionPorInstalacion) {  
        comisionPorInstalacion = _comisionPorInstalacion  
    }  
}
```

Esta solución es, en apariencia, equivalente a la solución inicial. Sin embargo, utiliza una estrategia problemática que denominamos **precálculo**: el valor de la comisión no se calcula al momento de consultarse -mensaje *comisionTotal()*-, si no que se va calculando antes de tiempo, a medida que se realizan las instalaciones. Ese desfase temporal es origen de errores: si justo antes de realizar el pago a Macacha, la empresa aumenta la comisión por instalación -por ejemplo, mediante el mensaje *macacha.comisionPorInstalacion(200)*- y luego se consulta por la comisión, el resultado será el valor que se precaculó con un número desactualizado.

Se pueden encontrar estrategias incorrectas para mantener las variables sincronizadas: recordar más cosas, actualizar varios atributos al cambiar la comisión por instalación, etc. Todos esos trucos que ensucian el código encuentran limitaciones a medida que el problema se complejiza: por ejemplo, si el valor de *comisionPorInstalacion* está en otro objeto y no en *macacha*, la sincronización se vuelve demasiado invasiva para el segundo objeto, con serias implicancias en la calidad del diseño. Todos esos problemas podrían evitarse si se calculara en el momento de uso.

Otra manera de visualizar el mismo problema es analizar la **dependencia entre atributos**. Los atributos deben ser independientes entre sí. Si ocurre que al modificar uno se debe modificar otro entonces no son independientes. En este caso, se ve que el atributo *comisionTotal* depende de *comisionPorInstalacion*, lo que da la pista de que *comision* es dependiente y, por lo tanto, no debería modelarse como atributo sino como un método que realice un cálculo.

**¡Cuidado!** Distinto sería el caso si la comisión que reciba Macacha por sus instalaciones dependiera del valor de *comisionPorInstalacion* **al momento de realizarse** la instalación. En ese caso, por decisión de las reglas de negocio, un aumento de *comisionPorInstalacion* se realiza para modificar las comisiones futuras de Macacha y no las ya realizadas. Ese problema es completamente diferente. En ese caso, las variables ya no son dependientes entre ellas porque la comisión **ya no depende del valor actual** de *comisionPorInstalacion*. Ir calculando la comisión a medida que se realizan las instalaciones es el momento correcto de utilizar el valor de dicho atributo, haciendo que la solución que se señalaba al inicio de esta sección como mala sea ahora correcta.